

École Centrale des Logiciels Libres et de Télécommunications

Projet 5 : Analyse du comportement de WebRTC face au cloisonnement des réseaux (NAT & Pare-feux): Cas d'un Chat Vidéo

Présenté par :

*Ousseynou KABA
El Yassine SALIM
Khalilou SYLLA
El Hadji GUEYE*

Sous la supervision :
M. NASRY

INTRODUCTION.....	3
I. Contexte et problématique.....	5
1. La communication temps réel sur le Web.....	5
2. Le cloisonnement des réseaux : NAT et pare-feux.....	5
II. Les mécanismes de traduction d'adresses (NAT) et leur diversité.....	6
1. Full cone NAT (NAT conique complet).....	6
2. Restricted Cone NAT.....	6
3. Port-Restricted Cone NAT:.....	6
4. Symmetric NAT:.....	7
III. STUN, TURN et ICE.....	7
1. STUN (Session Traversal Utilities for NAT).....	7
2. TURN (Traversal Using Relays around NAT).....	7
3. ICE (Interactive Connectivity Establishment).....	8
IV. Architecture de l'application support de l'étude.....	9
1. Composants principaux.....	9
2. Journalisation en temps réel des connexions.....	10
V. Mise en oeuvre.....	10
1. Configuration des fichier nécessaires.....	12
2. Teste du serveur ICE :.....	27
3. Teste de l'application en temps réel :.....	28
CONCLUSION.....	35

INTRODUCTION

La communication en temps réel sur Internet voix, vidéo, partage d'écran repose aujourd'hui très largement sur la technologie WebRTC (Web Real-Time Communication). Portée nativement par les navigateurs modernes, WebRTC permet d'établir des flux audio et vidéo directement entre deux terminaux, sans passer par un serveur intermédiaire pour le transport des médias eux-mêmes. Cette architecture, en principe pair-à-pair (P2P), promet une latence réduite, une charge serveur minimale et une meilleure qualité perçue par l'utilisateur.

Cependant, cette promesse d'une connexion directe se heurte à une réalité structurelle d'Internet : la très grande majorité des terminaux ne possèdent pas d'adresse IP publique directement joignable. Ils se trouvent derrière des équipements de traduction d'adresses (NAT: Network Address Translation) et des pare-feux, dont le rôle premier est de préserver l'espace d'adressage IPv4 restant et de protéger les réseaux locaux des connexions non sollicitées. Ces mécanismes, indispensables à la sécurité et à l'architecture actuelle d'Internet, constituent paradoxalement l'un des plus grands obstacles techniques à la mise en œuvre d'une communication P2P fiable.

C'est dans ce contexte que s'inscrit le projet présenté dans ce document : le développement d'une application de chat vidéo minimaliste, dotée d'un système d'affichage des logs de connexion en temps réel, permettant d'observer concrètement comment une session WebRTC parvient ou non à s'établir en fonction de l'environnement réseau des participants. L'objectif n'est pas seulement de produire une application fonctionnelle, mais d'en faire un instrument d'observation et d'analyse du comportement réel des protocoles de traversée de NAT que sont ICE, STUN et TURN.

La problématique centrale de cette étude peut ainsi être formulée de la manière suivante : comment les protocoles ICE, STUN et TURN permettent-ils de traverser les différents types de routeurs NAT, et dans quelle mesure leur efficacité varie-t-elle selon la nature du réseau traversé ?

Pour y répondre, ce document propose dans un premier temps un cadrage théorique du problème posé par le cloisonnement réseau, avant de détailler le fonctionnement des mécanismes NAT et des protocoles ICE, STUN et TURN qui permettent de le contourner. Il

présente ensuite l'architecture retenue pour l'application de chat vidéo et la méthodologie d'analyse académique mise en œuvre : des tests conduits sur des environnements réseau contrastés (Wi-Fi public, partage de connexion 4G/5G, réseau universitaire restreint), avec pour indicateur central le taux de réussite des connexions en P2P direct via STUN comparé au recours obligatoire au relais TURN. Une dernière partie discute les résultats attendus de cette analyse comparative et leurs implications pratiques, avant de conclure sur les enseignements généraux du projet.

I. Contexte et problématique

1. La communication temps réel sur le Web

WebRTC est un ensemble de normes et d'API standardisées par le W3C et l'IETF, intégré nativement dans les navigateurs (Chrome, Firefox, Edge, Safari) et dans de nombreuses applications mobiles. Il permet à deux agents applicatifs de négocier et d'établir un canal de communication multimédia sans plugin ni logiciel tiers. Les usages sont aujourd'hui omniprésents : visioconférence professionnelle, télémedecine, jeux en ligne, assistance à distance, ou encore chat vidéo grand public.

Contrairement aux modèles client-serveur classiques du Web, WebRTC vise une architecture pair-à-pair : une fois la session négociée, les flux audio et vidéo transitent directement entre les deux machines, sans repasser par un serveur applicatif. Cette caractéristique est à la fois la force et la principale difficulté technique de WebRTC.

2. Le cloisonnement des réseaux : NAT et pare-feux

Le NAT a été introduit pour pallier l'épuisement des adresses IPv4 : il permet à plusieurs machines d'un réseau local de partager une unique adresse IP publique en réécrivant dynamiquement les en-têtes des paquets sortants et en maintenant une table de correspondance (adresse/port privés ↔ adresse/port publics). Les pare-feux, quant à eux, filtrent le trafic entrant selon des règles de sécurité et bloquent par défaut toute connexion non initiée depuis l'intérieur du réseau.

Ces deux mécanismes ont une conséquence directe pour WebRTC : une machine A ne connaît pas son adresse IP publique réelle ni le port par lequel elle sera effectivement joignable depuis l'extérieur, et une machine B ne peut pas, a priori, initier une connexion entrante vers A. Établir une session P2P suppose donc de résoudre deux problèmes distincts : la découverte de l'adressage public utilisable (NAT traversal) et l'ouverture coordonnée d'un chemin bidirectionnel à travers les pare-feux (NAT hole punching).

II. Les mécanismes de traduction d'adresses (NAT) et leur diversité

Tous les NAT ne se comportent pas de la même manière face à une tentative de connexion entrante, ce qui explique que la traversée de NAT ne puisse pas reposer sur une solution unique. La classification historique, issue de la RFC 3489 puis affinée par la RFC 5389, distingue quatre grands comportements.

1. Full cone NAT (NAT conique complet)

Full Cone NAT (NAT conique complet) : Ce type de NAT établit une correspondance fixe entre l'adresse/port interne et l'adresse/port public. Une fois qu'un hôte interne a envoyé un paquet vers l'extérieur, tout hôte externe peut envoyer des paquets vers cette adresse publique, et ceux-ci seront transmis à l'hôte interne. Ce comportement est le plus permissif et facilite grandement les connexions P2P.

2. Restricted Cone NAT

Restricted Cone NAT (NAT conique restreint) : La correspondance d'adresse reste fixe, mais le filtrage est renforcé : un hôte externe ne peut envoyer un paquet vers l'hôte interne que si ce dernier a préalablement envoyé un paquet vers l'adresse IP de cet hôte externe. Le port source externe n'est pas vérifié.

3. Port-Restricted Cone NAT:

Port-Restricted Cone NAT (NAT conique restreint par port) : Variante plus stricte du précédent : l'hôte externe ne peut communiquer avec l'hôte interne que si celui-ci a préalablement envoyé un paquet vers la combinaison IP+port de l'hôte externe. Cette restriction au niveau du port rend la traversée plus délicate.

4. Symmetric NAT:

Symmetric NAT (NAT symétrique) : Ce type représente le défi majeur pour la traversée NAT. Contrairement aux NAT coniques, le NAT symétrique crée une nouvelle correspondance d'adresse/port pour chaque destination différente. Ainsi, lorsque l'hôte interne envoie un paquet vers le serveur A puis vers le serveur B, le NAT utilise deux adresses publiques différentes. Cette caractéristique rend impossible la prédiction de l'adresse publique qui sera utilisée pour une future connexion, et donc les techniques de « trou de poinçonnage » (hole punching) échouent lorsque les deux pairs sont derrière des NAT symétriques.

III. STUN, TURN et ICE

1. STUN (Session Traversal Utilities for NAT)

Défini par la RFC 5389, STUN est un protocole léger qui permet à un client, situé derrière un NAT, de découvrir l'adresse IP publique et le port sous lesquels il est vu depuis l'extérieur. Le client interroge un serveur STUN public ; celui-ci lui renvoie, dans sa réponse, le couple (adresse, port) tel qu'il l'a observé côté public. Cette information permet ensuite au client de communiquer cette adresse « réflexive » à son interlocuteur, afin que ce dernier tente une connexion directe vers ce point d'entrée.

STUN est peu coûteux en ressources car le serveur STUN n'intervient qu'au moment de la découverte d'adresse : il ne relaie ensuite aucun flux média. Sa limite majeure est qu'il ne fonctionne de manière fiable que face à des NAT de type cone (full, restricted ou port-restricted) : face à un NAT symétrique, l'adresse découverte via STUN n'est pas nécessairement celle qui sera utilisée pour joindre le pair distant, ce qui fait échouer la tentative de connexion directe.

2. TURN (Traversal Using Relays around NAT)

Défini par la RFC 5766, TURN prend le relais lorsque la connexion directe échoue. Contrairement à STUN, un serveur TURN ne se contente pas d'observer l'adresse du client : il agit comme un relais actif, retransmettant l'intégralité du flux média entre les deux pairs.

Chaque participant établit une allocation sur le serveur TURN, qui lui attribue une adresse et un port relais ; les paquets échangés transitent alors systématiquement par ce serveur.

TURN garantit un taux de réussite de connexion proche de 100 %, y compris dans les environnements les plus restrictifs (NAT symétrique combiné à un pare-feu strict), car il ne dépend pas de la capacité des deux NAT à laisser passer un trafic entrant non prévisible. Cette fiabilité a toutefois un coût : le serveur TURN doit disposer d'une bande passante suffisante pour relayer l'ensemble des flux, la latence est généralement plus élevée qu'en P2P direct, et l'opérateur du service supporte un coût d'infrastructure proportionnel au nombre de sessions relayées.

3. ICE (Interactive Connectivity Establishment)

ICE, normalisé par la RFC 8445, n'est pas un protocole de traversée de NAT à proprement parler, mais un cadre de négociation qui orchestre l'usage combiné de STUN et de TURN afin de sélectionner, parmi toutes les voies de communication possibles, la meilleure disponible. Concrètement, chaque agent WebRTC collecte un ensemble de « candidats » : son adresse locale, son adresse réflexive obtenue via un serveur STUN, et une adresse relais obtenue via un serveur TURN. Ces candidats sont échangés avec le pair distant via un canal de signalisation externe à WebRTC (généralement WebSocket ou Socket.io dans le cadre de ce projet).

Une fois les candidats échangés, ICE effectue des tests de connectivité systématiques (connectivity checks) entre chaque paire de candidats, par ordre de priorité décroissante : connexion locale directe, puis connexion P2P via l'adresse réflexive STUN, et enfin, en dernier recours, relais TURN. La paire retenue est celle qui fonctionne avec la priorité la plus élevée, ce qui garantit que WebRTC utilise systématiquement la voie la plus performante disponible plutôt que de basculer inutilement vers un relais coûteux.

Schéma de principe de la négociation ICE:

- ❖ Collecte des candidats locaux (host candidates)
- ❖ Interrogation d'un serveur STUN → candidats réflexifs (server-reflexive)
- ❖ Allocation sur un serveur TURN → candidats relais (relay candidates)
- ❖ Échange des candidats via le canal de signalisation (SDP offer/answer)

- ❖ Tests de connectivité par paires, du plus direct au plus coûteux
- ❖ Sélection de la meilleure paire fonctionnelle et établissement du flux média

IV. Architecture de l'application support de l'étude

Pour étudier concrètement le comportement décrit ci-dessus, le projet prévoit le développement d'une application de chat vidéo minimaliste, dont la fonction n'est pas de rivaliser avec des solutions commerciales, mais de servir de banc d'essai instrumenté pour observer la négociation ICE en conditions réelles.

1. Composants principaux

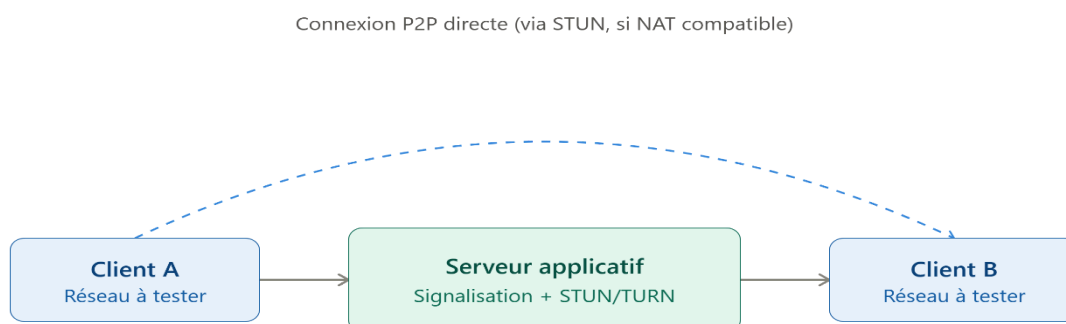
Composants	Rôles
PeerJS	Surcouche simplifiant la gestion des connexions WebRTC (RTCPeerConnection) côté client
Socket.io / WebSocket	Canal de signalisation: échange des offres/réponses SDP et des candidats ICE entre les deux navigateurs
Serveur Coturn	Implémentation du serveur STUN et TURN utilisé pour la découverte d'adresse et le relais média
HTTPS	Sécurisation du transport, condition requise par les navigateurs pour autoriser l'accès caméra/micro
PostgreSQL	Stockage structuré des sessions et des journaux de connexion (métadonnées, horodatage, type de candidat retenu)
MinIO	Stockage objet, réservé à d'éventuels exports de logs ou d'enregistrements

2. Journalisation en temps réel des connexions

La spécificité fonctionnelle de l'application est son système de logs de connexion affichés en temps réel. À chaque étape de la négociation ICE, l'application capture et affiche : le type de candidat proposé (host, server-réflexive ou relay), l'état de la connexion (checking, connected, failed), le type de la paire de candidats finalement sélectionnée, ainsi que le délai écoulé entre l'initiation de l'appel et l'établissement effectif du flux média.

Cette instrumentation transforme l'application en un outil de mesure : elle permet, pour chaque test réalisé sur un réseau donné, de déterminer sans ambiguïté si la connexion a été établie en P2P direct (candidat server-réflexive retenu) ou si elle a nécessité un relais (candidat relay retenu), et de mesurer le temps de connexion associé à chaque cas.

V. Mise en oeuvre

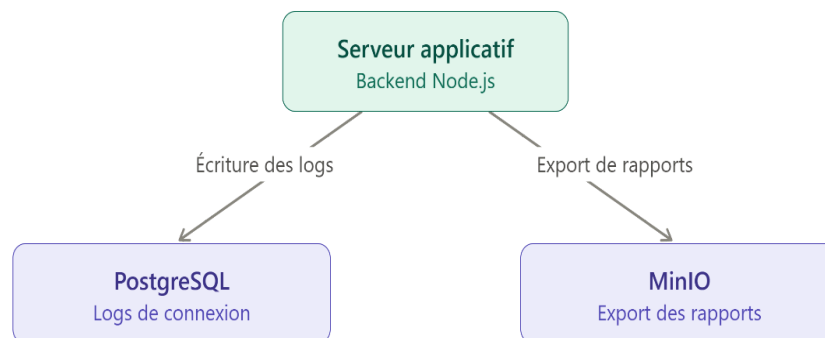


Ce que montre ce schéma :

- **Client A / Client B** les navigateurs, chacun derrière un routeur NAT différent selon le réseau testé (Wi-Fi public, partage 4G/5G, réseau universitaire bridé).
- **Serveur applicatif** un seul processus Node.js qui héberge à la fois le canal de signalisation (Socket.io pour la gestion de salle/logs, PeerJS pour l'échange SDP/ICE) et le service **Coturn** (STUN + TURN). Le tout doit tourner sur une machine avec une

IP publique fixe, c'est indispensable : un serveur Coturn derrière un NAT ne peut pas servir de relais, donc pour ton protocole de test (mesurer le taux de succès P2P des *clients*), le serveur lui-même doit être hors de toute contrainte NAT.

- **Le trait pointillé (P2P direct)** : c'est le cas où STUN suffit à découvrir une paire de candidats *server-réflexive* utilisables directement entre A et B, sans passer les données média par le serveur.
- Si le NAT est trop restrictif (typiquement NAT symétrique des deux côtés), ICE échoue à établir une paire directe et le flux média est **relayé** par Coturn c'est le cas que tu veux justement détecter et compter dans tes log.



Répartition des rôles :

- **PostgreSQL** : la base structurée pour l'analyse quantitative : chaque tentative de connexion devient une ligne avec l'horodatage, le réseau testé (Wi-Fi public / 4G-5G / campus), le type de candidat ICE retenu (host, srflx, relay), l'état final (succès/échec) et éventuellement la latence (RTT). C'est cette table qui te permettra de calculer directement ton taux de réussite P2P vs relais TURN par type de réseau.
- **MinIO** : stockage objet pour tout ce qui est volumineux ou peu structuré : dump JSON complet du cycle ICE gathering d'une session, éventuellement un export PDF/CSV généré pour le rapport académique. Séparer ça de PostgreSQL évite d'alourdir la base avec des blobs.

1. Configuration des fichier nécessaires

```
root@elyassine-VirtualBox:/home/elyassine# mkdir webrtc-lab && cd webrtc-lab
mkdir -p public views certs
root@elyassine-VirtualBox:/home/elyassine/webrtc-lab#
```

Cette capture montre la création de l'arborescence du projet (dossiers public, views, certs) puis le passage dans le répertoire webrtc-lab en préparation de l'installation des dépendances.

```
root@elyassine-VirtualBox:/home/elyassine# sudo -u postgres createdb webrtc_lab
sudo -u postgres psql -d webrtc_lab << 'EOF'
CREATE TABLE IF NOT EXISTS connection_logs (
  id          BIGSERIAL PRIMARY KEY,
  session_id  TEXT,
  peer_id     TEXT,
  network_type TEXT,
  event_type  TEXT NOT NULL,
  candidate_type TEXT,
  ice_state   TEXT,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS idx_logs_session ON connection_logs(session_id);
CREATE INDEX IF NOT EXISTS idx_logs_created ON connection_logs(created_at DESC);
EOF
could not change directory to "/home/elyassine": Permission non accordée
could not change directory to "/home/elyassine": Permission non accordée
CREATE TABLE
CREATE INDEX
CREATE INDEX
root@elyassine-VirtualBox:/home/elyassine#
```

Création de la base PostgreSQL webrtc_lab et de la table connection_logs (identifiant, session, pair, type de réseau, type de candidat ICE, état, horodatage), avec deux index pour accélérer les requêtes par session et par date. Les lignes « Permission non accordée » sont des avertissements sans conséquence liés au répertoire courant de l'utilisateur postgres : les instructions CREATE TABLE/INDEX qui suivent aboutissent bien.

```
root@elyassine-VirtualBox:/home/elyassine/webrtc-lab# npm install
added 113 packages, and audited 114 packages in 39s

18 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
root@elyassine-VirtualBox:/home/elyassine/webrtc-lab# cd /home/elyassine/webrtc-lab
```

Installation des dépendances Node.js du projet via npm install (113 paquets, aucune vulnérabilité détectée).

Server.js

```

require('dotenv').config();

const fs = require('fs');
const https = require('https');
const path = require('path');
const express = require('express');
const { Server } = require('socket.io');
const { Pool } = require('pg');

const app = express();

app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));
app.use(express.static(path.join(__dirname, 'public')));
app.use(express.json());

const server = https.createServer(
  {
    key: fs.readFileSync(path.join(__dirname, 'certs', 'key.pem')),
    cert: fs.readFileSync(path.join(__dirname, 'certs', 'cert.pem')),
  },
  app
);

const io = new Server(server);

const pool = new Pool({

```

Début de server.js : chargement des variables d'environnement, imports (fs, https, express, Socket.io, pg), configuration d'Express (moteur de vues EJS, fichiers statiques, JSON), puis création du serveur HTTPS à partir des certificats locaux (certs/key.pem et cert.pem) et initialisation du serveur [Socket.io](#).

```

const pool = new Pool({
  host: process.env.PGHOST,
  port: Number(process.env.PGPORT) || 5432,
  user: process.env.PGUSER,
  password: process.env.PGPASSWORD,
  database: process.env.PGDATABASE,
});

let dbReady = false;
pool.query('SELECT 1')
  .then(() => { dbReady = true; console.log(' PostgreSQL connecté'); })
  .catch((err) => console.warn('PostgreSQL indisponible :', err.message));

async function saveLog(entry) {
  if (!dbReady) return;
  try {
    await pool.query(
      `INSERT INTO connection_logs
      (session_id, peer_id, network_type, event_type, candidate_type, ice_state)
      VALUES ($1,$2,$3,$4,$5,$6)`,
      [
        entry.sessionId || null,
        entry.peerId || null,
        entry.networkType || null,
        entry.eventType || null,
        entry.candidateType || null,

```

Configuration du pool de connexions PostgreSQL à partir des variables d'environnement, test de connexion (dbReady), puis définition de la fonction saveLog(), qui insère chaque évènement de connexion dans la table connection_logs.

```
    entry.peerId || null,
    entry.networkType || null,
    entry.eventType || null,
    entry.candidateType || null,
    entry.iceState || null,
  ]
);
} catch (err) {
  console.error('Erreur insertion log:', err.message);
}
}

const rooms = new Map();

io.on('connection', (socket) => {
  console.log('Socket connecté :', socket.id);

  socket.on('join', ({ roomId, networkType }) => {
    socket.data.roomId = roomId;
    socket.data.networkType = networkType;
    socket.join(roomId);

    if (!rooms.has(roomId)) rooms.set(roomId, []);
    const others = rooms.get(roomId).filter((p) => p.socketId !== socket.id);
    socket.emit('peers-in-room', others.map((p) => p.socketId));

    rooms.get(roomId).push({ socketId: socket.id, networkType });
  });
});
```

Structure de gestion des salles (Map rooms) et gestionnaire de connexion Socket.io : à l'évènement « join », le serveur rattache le socket à la salle demandée, notifie les pairs déjà présents et journalise l'évènement via saveLog().

```
socket.emit('peers-in-room', others.map((p) => p.socketId));

rooms.get(roomId).push({ socketId: socket.id, networkType });
socket.to(roomId).emit('peer-joined', { socketId: socket.id });

saveLog({ sessionId: roomId, peerId: socket.id, networkType, eventType: 'join' });
});

socket.on('signal', ({ to, data }) => {
  io.to(to).emit('signal', { from: socket.id, data });
});

socket.on('ice-log', (entry) => {
  const enriched = {
    sessionId: socket.data.roomId,
    peerId: socket.id,
    networkType: socket.data.networkType,
    eventType: entry.eventType,
    candidateType: entry.candidateType,
    iceState: entry.iceState,
  };
  saveLog(enriched);
  io.to(socket.data.roomId).emit('log-broadcast', enriched);
});

socket.on('disconnect', () => {
  const { roomId } = socket.data;
  rooms.get(roomId).push({ socketId: socket.id, networkType });
});
```

Relais de la signalisation WebRTC (événement « signal », transmis tel quel au destinataire) et gestionnaire « ice-log », qui enrichit chaque entrée de journal côté client avec l'identifiant de salle et de pair avant de la rediffuser à tous les participants et de l'enregistrer en base.

```
socket.on('disconnect', () => {
  const { roomId } = socket.data;
  if (roomId && rooms.has(roomId)) {
    rooms.set(roomId, rooms.get(roomId).filter((p) => p.socketId !== socket.id));
    if (rooms.get(roomId).length === 0) rooms.delete(roomId);
    socket.to(roomId).emit('peer-left', { socketId: socket.id });
  }
  console.log('Socket déconnecté :', socket.id);
});
});

app.get('/', (req, res) => {
  res.render('index', {
    turnHost: process.env.TURN_HOST || '',
    turnUser: process.env.TURN_USER || '',
    turnPass: process.env.TURN_PASS || ''
  });
});

app.get('/health', async (req, res) => {
  res.json({ ok: true, db: dbReady, rooms: rooms.size });
});

const PORT = process.env.PORT || 4000;
server.listen(PORT, () => {
  console.log(`\n🟢https://localhost:${PORT}\n`);
});
```

Nettoyage à la déconnexion (retrait de la salle, notification « peer-left » aux autres participants), route principale « / » qui injecte les identifiants TURN dans la page EJS, endpoint « /health » pour vérifier l'état du serveur et de la base, puis démarrage du serveur HTTPS sur le port configuré.

Index.ejs

```

GNU nano 6.2                                     index.ejs *
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>WebRTC NAT Lab</title>
  <link rel="stylesheet" href="/style.css" />
</head>
<body>
  <h1>WebRTC NAT Lab</h1>
  <p class="subtitle">Analyse du comportement ICE / STUN / TURN selon le réseau</p>

  <div id="setup">
    <input id="roomId" type="text" placeholder="ID de salle (ex: demo)" value="demo" />
    <select id="networkType">
      <option value="lan">LAN (même réseau)</option>
      <option value="wifi">Wi-Fi</option>
      <option value="4g">4G / Mobile</option>
      <option value="vpn">VPN</option>
      <option value="nat-symmetric">NAT symétrique</option>
      <option value="unknown">Inconnu</option>
    </select>
    <button id="joinBtn">Rejoindre</button>
  </div>

  <div id="controls" style="display:none;">
    <button id="toggleMic"> Couper micro</button>

```

Début de index.ejs : en-tête HTML, titre de l'application, puis formulaire de connexion avec un champ d'identifiant de salle et un menu déroulant de type de réseau (LAN, Wi-Fi, 4G/Mobile, VPN, NAT symétrique, Inconnu) — cette valeur est saisie manuellement par le testeur.

```

  <label style="margin-left:1rem;">
    Volume distant
    <input id="remoteVolume" type="range" min="0" max="1" step="0.05" value="1" />
  </label>
</div>

<div id="videos">
  <video id="localVideo" autoplay playsinline muted></video>
  <video id="remoteVideo" autoplay playsinline></video>
</div>

<h2>Journal des événements ICE</h2>
<ul id="logPanel"></ul>

<script>
  window.__ICE_CONFIG__ = {
    turnHost: <%- JSON.stringify(turnHost || '') %>,
    turnUser: <%- JSON.stringify(turnUser || '') %>,
    turnPass: <%- JSON.stringify(turnPass || '') %>
  };

```

Contrôles de l'appel (couper micro, volume distant), zones vidéo locale et distante, puis panneau « Journal des événements ICE ». Le script injecte côté serveur les identifiants du serveur TURN (hôte, utilisateur, mot de passe) dans une variable JavaScript globale utilisée ensuite par client.js

```
</script>
<script src="/socket.io/socket.io.js"></script>
<script src="/client.js"></script>
</body>
</html>
```

Clients js

```
GNU nano 6.2 client.js
const socket = io();

let localStream = null;
let roomId = null;
let networkType = null;
let joined = false;
const peerConnections = new Map();

const logPanel = document.getElementById('logPanel');
const localVideo = document.getElementById('localVideo');
const remoteVideo = document.getElementById('remoteVideo');
const controls = document.getElementById('controls');
const joinBtn = document.getElementById('joinBtn');

function logEvent(entry) {
  const li = document.createElement('li');
  li.textContent =
    `[${new Date().toLocaleTimeString()}] ${entry.eventType}` +
    (entry.candidateType ? ` - ${entry.candidateType}` : '') +
    (entry.iceState ? ` - ${entry.iceState}` : '');
  logPanel.prepend(li);
  if (joined) socket.emit('ice-log', entry);
}

function candidateType(str) {
  const m = /typ (\w+)/.exec(str || '');
  return m ? m[1] : null;
}
```

Début de client.js : connexion au serveur Socket.io, récupération des éléments du DOM, fonction logEvent() qui affiche chaque événement dans le journal et le transmet au serveur, fonction utilitaire candidateType() qui extrait le type de candidat ICE (host/srflx/relay) à partir de sa description, puis début de buildIceServers().

```

    return M ? M[1] : null;
}

function buildIceServers() {
    const { turnHost, turnUser, turnPass } = window.__ICE_CONFIG__ || {};
    if (turnHost) {
        return [
            { urls: `stun:${turnHost}:3478` },
            { urls: `turn:${turnHost}:3478`, username: turnUser, credential: turnPass },
            { urls: `turns:${turnHost}:5349`, username: turnUser, credential: turnPass },
        ];
    }
    return [
        { urls: 'stun:stun.l.google.com:19302' },
        { urls: 'stun:stun1.l.google.com:19302' },
    ];
}

function createPeerConnection(remoteSocketId) {
    const pc = new RTCPeerConnection({ iceServers: buildIceServers() });

    localStream.getTracks().forEach((track) => pc.addTrack(track, localStream));

    pc.onicecandidate = (event) => {
        if (event.candidate) {
            logEvent({
                eventType: 'ice-candidate',

```

Suite de buildIceServers() : si un hôte TURN est configuré, la fonction ne renvoie que les URLs pointant vers ce serveur (STUN, TURN/UDP et TURN/TLS) ; sinon elle retombe sur les serveurs STUN publics de Google. Ensuite, createPeerConnection() crée la RTCPeerConnection avec ces serveurs ICE et y ajoute les pistes audio/vidéo locales, puis le gestionnaire onicecandidate commence à être défini.

```

        eventType: 'ice-candidate',
        candidateType: candidateType(event.candidate.candidate),
    });
    socket.emit('signal', {
        to: remoteSocketId,
        data: { type: 'ice', candidate: event.candidate },
    });
} else {
    logEvent({ eventType: 'ice-candidate', candidateType: 'end' });
}
};

pc.oniceconnectionstatechange = () => {
    logEvent({ eventType: 'ice-state', iceState: pc.iceConnectionState });
};

pc.onconnectionstatechange = () => {
    logEvent({ eventType: 'pc-state', iceState: pc.connectionState });
};

pc.ontrack = (event) => {
    const [stream] = event.streams;
    remoteVideo.srcObject = stream;
    remoteVideo.play().catch(() => {
        if (document.getElementById('unmuteBtn')) return;
        const btn = document.createElement('button');
        btn.id = 'unmuteBtn';

```

Suite du gestionnaire onicecandidate (envoi de chaque candidat au pair distant via le canal de signalisation et journalisation de son type), suivie des gestionnaires oniceconnectionstatechange et onconnectionstatechange qui journalisent les changements d'état, puis début du gestionnaire ontrack qui attache le flux distant reçu à l'élément vidéo correspondant.

```
        btn.textContent = ' Activer le son distant';
        btn.onclick = () => {
            remoteVideo.muted = false;
            remoteVideo.play();
            btn.remove();
        };
        document.body.appendChild(btn);
    });
};

peerConnections.set(remoteSocketId, pc);
return pc;
}

async function callPeer(remoteSocketId) {
    const pc = createPeerConnection(remoteSocketId);
    const offer = await pc.createOffer();
    await pc.setLocalDescription(offer);
    socket.emit('signal', {
        to: remoteSocketId,
        data: { type: 'offer', sdp: pc.localDescription },
    });
    logEvent({ eventType: 'offer-sent', iceState: remoteSocketId.slice(0, 6) });
}

async function handleSignal({ from, data }) {
    let pc = peerConnections.get(from);
```

Suite du gestionnaire ontrack : si la lecture automatique du son est bloquée par le navigateur, un bouton « Activer le son distant » est créé pour permettre à l'utilisateur de la déclencher manuellement.

```

if (data.type === 'offer') {
  if (!pc) pc = createPeerConnection(from);
  await pc.setRemoteDescription(new RTCSessionDescription(data.sdp));
  const answer = await pc.createAnswer();
  await pc.setLocalDescription(answer);
  socket.emit('signal', {
    to: from,
    data: { type: 'answer', sdp: pc.localDescription },
  });
  logEvent({ eventType: 'offer-received', iceState: from.slice(0, 6) });
} else if (data.type === 'answer') {
  if (pc) {
    await pc.setRemoteDescription(new RTCSessionDescription(data.sdp));
    logEvent({ eventType: 'answer-received', iceState: from.slice(0, 6) });
  }
} else if (data.type === 'ice') {
  if (pc) {
    try { await pc.addIceCandidate(new RTCIceCandidate(data.candidate)); }
    catch (e) { console.warn('addIceCandidate', e); }
  }
}
}

socket.on('signal', handleSignal);

socket.on('peers-in-room', (others) => {
  logEvent({ eventType: 'peers-in-room', iceState: others.length });
});

```

Stockage des connexions pairs dans une Map, fonction callPeer() qui crée une offre SDP et l'envoi au pair distant, puis début de handleSignal(), qui traite les messages de signalisation entrants.

```

others.forEach((id) => callPeer(id));
});

socket.on('peer-joined', ({ socketId }) => {
  logEvent({ eventType: 'peer-joined', iceState: socketId.slice(0, 6) });
});

socket.on('peer-left', ({ socketId }) => {
  logEvent({ eventType: 'peer-left', iceState: socketId.slice(0, 6) });
  const pc = peerConnections.get(socketId);
  if (pc) { pc.close(); peerConnections.delete(socketId); }
  if (peerConnections.size === 0) remoteVideo.srcObject = null;
});

socket.on('log-broadcast', (entry) => {
  if (entry.peerId && entry.peerId !== socket.id) {
    const li = document.createElement('li');
    li.textContent = `[distant] ${entry.eventType} +` +
      (entry.candidateType ? ` - ${entry.candidateType}` : '');
    logPanel.prepend(li);
  }
});

document.getElementById('toggleMic').addEventListener('click', (e) => {
  if (!localStream) return;
  const track = localStream.getAudioTracks()[0];
  if (!track) return;
});

```

Suite de `handleSignal()` : traitement des messages de type offre, réponse et candidat ICE selon le cas. En dessous, les écouteurs `socket.on('signal')` et `socket.on('peers-in-room')` déclenchent respectivement le traitement des messages reçus et l'appel de chaque pair déjà présent dans la salle.

```
track.enabled = !track.enabled;
e.target.textContent = track.enabled ? 'Couper le micro' : 'Activer le micro';
});

document.getElementById('toggleCam').addEventListener('click', (e) => {
  if (!localStream) return;
  const track = localStream.getVideoTracks()[0];
  if (!track) return;
  track.enabled = !track.enabled;
  e.target.textContent = track.enabled ? 'Couper la caméra' : 'Activer la caméra';
});

document.getElementById('remoteVolume').addEventListener('input', (e) => {
  remoteVideo.volume = parseFloat(e.target.value);
});

joinBtn.addEventListener('click', async () => {
  if (joined) return;

  roomId = document.getElementById('roomId').value.trim();
  networkType = document.getElementById('networkType').value;
  if (!roomId) { alert('ID de salle requis'); return; }

  try {
    localStream = await navigator.mediaDevices.getUserMedia({
      video: true,
      audio: true,
    });
```

Gestionnaires « peer-joined » et « peer-left » qui journalisent l'arrivée/le départ d'un pair et referment la connexion correspondante, gestionnaire « log-broadcast » qui affiche dans le journal les événements provenant du pair distant, puis écouteur du bouton de coupure du micro.

```

} catch (err) {
  alert(`Erreur caméra/micro : ${err.name} - ${err.message}`);
  console.error(err);
  return;
}

localVideo.srcObject = localStream;
controls.style.display = 'flex';
joined = true;

socket.emit('join', { roomId, networkType });
logEvent({ eventType: 'join-sent', iceState: roomId });
});

```

Écouteur du bouton de coupure de caméra, écouteur du curseur de volume distant, et enfin le gestionnaire de clic du bouton « Rejoindre » : demande d'accès à la caméra/au micro via `getUserMedia()`, affichage du flux local et envoi de l'évènement « join » au serveur.

Style.css

```

GNU nano 6.2 style.css
* { box-sizing: border-box; }
body {
  font-family: system-ui, -apple-system, sans-serif;
  max-width: 1000px;
  margin: 2rem auto;
  padding: 0 1rem;
  background: #0f172a;
  color: #e2e8f0;
}
h1 { margin-bottom: 0.25rem; }
h2 { margin-top: 2rem; }
.subtitle { color: #94a3b8; margin-top: 0; }
#setup { display: flex; gap: 0.5rem; margin: 1rem 0; flex-wrap: wrap; }
#setup input, #setup select, #setup button {
  padding: 0.5rem 0.75rem; border-radius: 6px;
  border: 1px solid #334155; background: #1e293b; color: #e2e8f0; font-size: 0.95rem;
}
#setup button { background: #2563eb; border: none; cursor: pointer; font-weight: 600; }
#setup button:hover { background: #1d4ed8; }
#controls {
  display: flex; gap: 0.75rem; align-items: center; flex-wrap: wrap;
  margin: 0.75rem 0; padding: 0.75rem;
  background: #1e293b; border: 1px solid #334155; border-radius: 8px;
}
#controls button {
  padding: 0.4rem 0.8rem; border-radius: 6px;
  border: 1px solid #334155; background: #0f172a; color: #e2e8f0; cursor: pointer;
}

```

Début de `style.css` : réinitialisation de base, habillage sombre du corps de page, puis mise en forme du bloc `#setup` (champ, menu déroulant et bouton de connexion en disposition flexible).

```

#setup button { background: #2563eb; border: none; cursor: pointer; font-weight: 600; }
#setup button:hover { background: #1d4ed8; }
#controls {
  display: flex; gap: 0.75rem; align-items: center; flex-wrap: wrap;
  margin: 0.75rem 0; padding: 0.75rem;
  background: #1e293b; border: 1px solid #334155; border-radius: 8px;
}
#controls button {
  padding: 0.4rem 0.8rem; border-radius: 6px;
  border: 1px solid #334155; background: #0f172a; color: #e2e8f0; cursor: pointer;
}
#controls button:hover { background: #1d4ed8; }
#videos { display: flex; gap: 1rem; margin: 1rem 0; flex-wrap: wrap; }
video {
  width: 48%; min-width: 240px; aspect-ratio: 16 / 9;
  background: #000; border-radius: 8px; object-fit: cover;
}
#logPanel {
  list-style: none; padding: 0.5rem; max-height: 320px; overflow-y: auto;
  border: 1px solid #334155; border-radius: 8px;
  font-family: ui-monospace, monospace; font-size: 0.82rem; background: #111827;
}
#logPanel li { padding: 0.15rem 0; border-bottom: 1px solid #1e293b; }
@media (max-width: 640px) { video { width: 100%; } }

```

Suite de style.css : mise en forme des contrôles d'appel, disposition des zones vidéo (ratio 16/9), style du panneau de journal (police à chasse fixe, défilement vertical), et règle média pour adapter l'affichage sur petit écran.

```

root@elyassine-VirtualBox:/home/elyassine/webrtc-lab# node server.js

https://localhost:4000

PostgreSQL connecté
Socket connecté : IDpCQfd0xAAeX3L_AAAA
Socket déconnecté : IDpCQfd0xAAeX3L_AAAA
Socket connecté : PyttuEe5jUKidsjzAAAB
Socket connecté : XAnYI0wxRPQtActZAAAC
Socket connecté : 3ceczl9XF0BvkehTAAAD
Socket déconnecté : 3ceczl9XF0BvkehTAAAD
Socket déconnecté : XAnYI0wxRPQtActZAAAC
Socket connecté : HrXM5CGXA3bY1_M4AAAE
Socket connecté : V430JxqDEw02-H5PAAAF
Socket déconnecté : V430JxqDEw02-H5PAAAF
Socket déconnecté : HrXM5CGXA3bY1_M4AAAE
Socket connecté : KbncFNXNe-2lcQy0AAAG
Socket déconnecté : KbncFNXNe-2lcQy0AAAG
Socket connecté : zdIJ_wupJLt6MxmPAAAH
Socket déconnecté : PyttuEe5jUKidsjzAAAB
Socket connecté : L6SBYyI2IdP_HyTCAAAI
Socket connecté : SDFLS_gsgZV8tdHkAAAJ
Socket déconnecté : SDFLS_gsgZV8tdHkAAAJ
Socket connecté : z_oz_XXqvWLTmKn6AAAK
Socket déconnecté : z_oz_XXqvWLTmKn6AAAK
Socket connecté : 29UbepS3ZrwrSP0SAAAL
Socket connecté : Y63d9aWNfo5jSXARAAAM
Socket déconnecté : 29UbepS3ZrwrSP0SAAAL
Socket déconnecté : L6SBYyI2IdP_HyTCAAAI
Socket déconnecté : Y63d9aWNfo5jSXARAAAM

```

Extrait de la console du serveur au lancement (node server.js) : connexion à PostgreSQL confirmée, puis une longue série de connexions/déconnexions de sockets rapprochées, probablement liée à des rechargements de page répétés pendant les essais plutôt qu'à de véritables sessions d'appel.

Création du fichier turnserver.conf

```
GNU nano 6.2                                     turnserver.conf *
# Port d'écoute standard pour STUN/TURN
listening-port=3478

# Port pour TURN over TLS (optionnel mais recommandé si tu as des certificats valides)
tls-listening-port=5349

# Écoute sur toutes les interfaces
listening-ip=0.0.0.0

# IMPORTANT: Adresse IP publique de ton serveur.
# Les navigateurs l'utiliseront pour se connecter au TURN.
external-ip=192.168.1.92

# Plage de ports UDP pour le relais des médias (doit être ouverte dans le firewall)
min-port=49152
max-port=65535

# Domaine (realm) utilisé pour l'authentification
realm=webrtc.lab

# Authentification par identifiants statiques (simple pour un TP)
lt-cred-mech
user=testuser:TestPassword123

# Sécurité de base
fingerprint
no-multicast-peers
```

Première version de turnserver.conf : ports d'écoute STUN/TURN (3478) et TURN over TLS (5349), adresse IP externe déclarée, plage de ports UDP pour le relais média, domaine d'authentification et identifiants statiques.

```
# IMPORTANT: Adresse IP publique de ton serveur.
# Les navigateurs l'utiliseront pour se connecter au TURN.
external-ip=192.168.1.92

# Plage de ports UDP pour le relais des médias (doit être ouverte dans le firewall)
min-port=49152
max-port=65535

# Domaine (realm) utilisé pour l'authentification
realm=webrtc.lab

# Authentification par identifiants statiques (simple pour un TP)
lt-cred-mech
user=testuser:Passer123

# Sécurité de base
fingerprint
no-multicast-peers
no-cli

# Logs (utile pour le debug)
log-file=stdout
verbose
```

Version finalisée de turnserver.conf, avec mot de passe mis à jour et deux options supplémentaires : no-cli (désactive l'interface d'administration à distance) et la journalisation en sortie standard (log-file=stdout, verbose).

Création du fichier docker-compose.yml

```
GNU nano 6.2                                docker-compose.yml *
version: "3.9"

services:
  coturn:
    image: coturn/coturn:4.6.2
    container_name: webrtc-coturn
    restart: unless-stopped
    # IMPORTANT: Sur Linux, utiliser network_mode:host est fortement recommandé
    # pour que Coturn voie les vraies IP des clients et gère correctement les ports de relais.
    network_mode: host
    volumes:
      - ./turnserver.conf:/etc/coturn/turnserver.conf:ro
    command:
      - "-c"
      - "/etc/coturn/turnserver.conf"
```

Fichier docker-compose.yml définissant le service Coturn : image officielle coturn/coturn, mode réseau « host » (nécessaire pour que Coturn voie les vraies adresses IP des clients), montage du fichier de configuration en lecture seule et commande de démarrage.

Lancer le conteneur docker à partir du fichier docker-compose

```
root@elyassine-VirtualBox:/home/elyassine/webrtc-lab# docker run -d --network=host --name=webrtc-coturn \
-v /home/elyassine/webrtc-lab/turnserver.conf:/etc/coturn/turnserver.conf \
coturn/coturn -c /etc/coturn/turnserver.conf
Unable to find image 'coturn/coturn:latest' locally
latest: Pulling from coturn/coturn
34c79ea33466: Pull complete
2e19782d7109: Pull complete
66c83ee8316c: Pull complete
603afa840a83: Pull complete
Digest: sha256:bbefd3e1fdcdc0d58770fe01b581fd8b00d9f3a5580d00acb77cf719a6bc78e3
Status: Downloaded newer image for coturn/coturn:latest
a3da589c7d1c3ec51a33cf245a6ae0275f1deaad49ae682794c9812b1f462ef5
root@elyassine-VirtualBox:/home/elyassine/webrtc-lab#
```

Lancement effectif du conteneur Coturn via la commande docker run (et non docker-compose up, malgré le fichier docker-compose.yml défini juste avant) : téléchargement de l'image officielle puis démarrage du conteneur webrtc-coturn.

Configuration de fichier .env

```
GNU nano 6.2 .env *
PORT=4000
PGHOST=localhost
PGPORT=5432
PGUSER=postgres
PGPASSWORD=postgres
PGDATABASE=webrtc_lab
TURN_HOST=192.168.1.92
TURN_USER=testuser
TURN_PASS=Passer123
```

Fichier .env du projet : port d'écoute de l'application, paramètres de connexion à PostgreSQL, et coordonnées du serveur TURN (hôte, utilisateur, mot de passe) reprises telles quelles dans turnserver.conf.

2. Teste du serveur ICE :

Interface de l'outil de test « Trickle ICE » : liste des serveurs STUN/TURN configurés (STUN public Google et le serveur TURN du projet) avec les identifiants associés, avant de lancer la collecte des candidats.

Serveurs ICE

stun:stun.l.google.com:19302

URI STUN ou TURN :

Nom d'utilisateur
TURN :

Mot de passe TURN :

Ajouter un serveur

Supprimer le
serveur

Réinitialiser les
paramètres par
défaut

Interface de l'outil de test « Trickle ICE » : liste des serveurs STUN/TURN configurés (STUN public Google et le serveur TURN du projet) avec les identifiants associés, avant de lancer la collecte des candidats.

Options ICE

Valeur d'IceTransports : ☒ tous ☐ relais

☐ Obtenir les autorisations d'utilisation du microphone et de la caméra

Temps	Taper Fondation	Protocole	Adresse	Port	Priorité	URL (le cas échéant)	RelayProtocol (le cas échéant)
0,007	hôte 3382423399	udp	653b83c9-84f1-4d4f-bb13-43cbf9327f61.local	62556	126 30 255		
0,008	srflx 3771829553	udp	192.168.1.187	62556	100 30 255	stun:192.168.1.92:3478	
0,073	srflx 3771829553	udp	154.125.126.42	62556	100 30 255	stun.stun.l.google.com:19302	
0,253	relais 2601847750	udp	192.168.1.92	61328	3 31 255	turn:192.168.1.92:3478?transport=udp	
0,254	Fait						

Rassembler les candidats

Tableau des candidats ICE collectés par l'outil : un candidat host local, deux candidats srflx (l'un renvoyé par le serveur Coturn du projet en adresse privée, l'autre par le STUN public de Google en adresse publique réelle) et un candidat relay via le serveur TURN du projet.

3. Teste de l'application en temps réel :

Non sécurisé https://192.168.1.92:4000



Votre connexion n'est pas privée

Des pirates informatiques tentent peut-être de voler vos informations sur **192.168.1.92** (mots de passe, messages ou cartes de crédit, par exemple). [En savoir plus sur cet avertissement](#)

NET::ERR_CERT_AUTHORITY_INVALID

Masquer les paramètres avancés

Revenir en lieu sûr

Premier avertissement du navigateur lors de l'accès à l'application en HTTPS : la connexion est signalée comme non sécurisée car le certificat est auto-signé.

Non sécurisé <https://192.168.1.92:4000>



Votre connexion n'est pas privée

Des pirates informatiques tentent peut-être de voler vos informations sur **192.168.1.92** (mots de passe, messages ou cartes de crédit, par exemple). [En savoir plus sur cet avertissement](#)

NET::ERR_CERT_AUTHORITY_INVALID

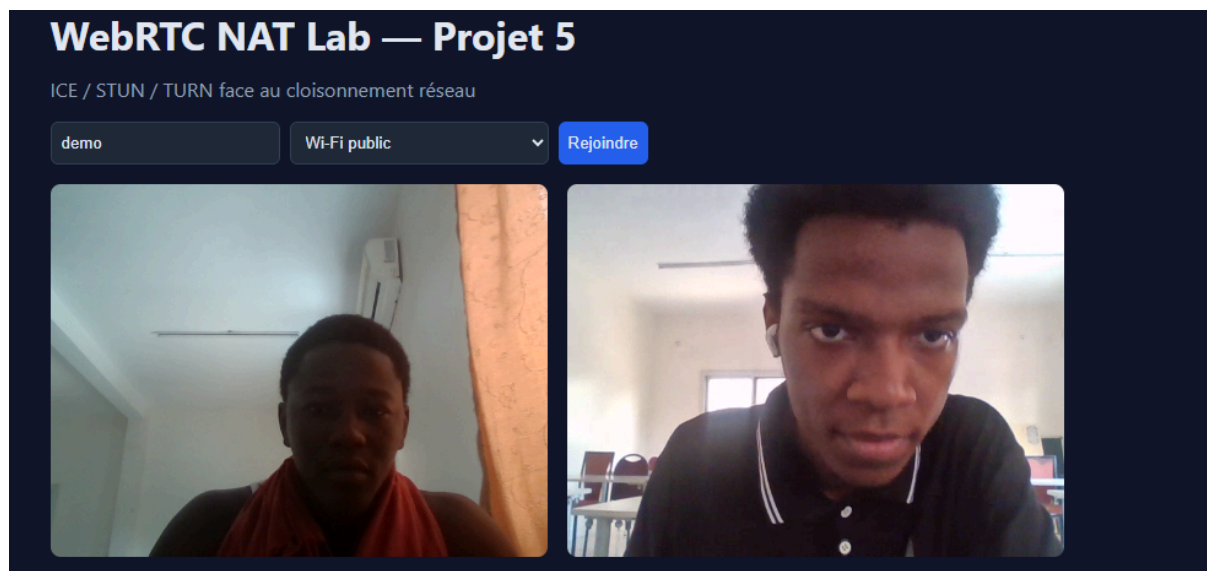
Masquer les paramètres avancés

Revenir en lieu sûr

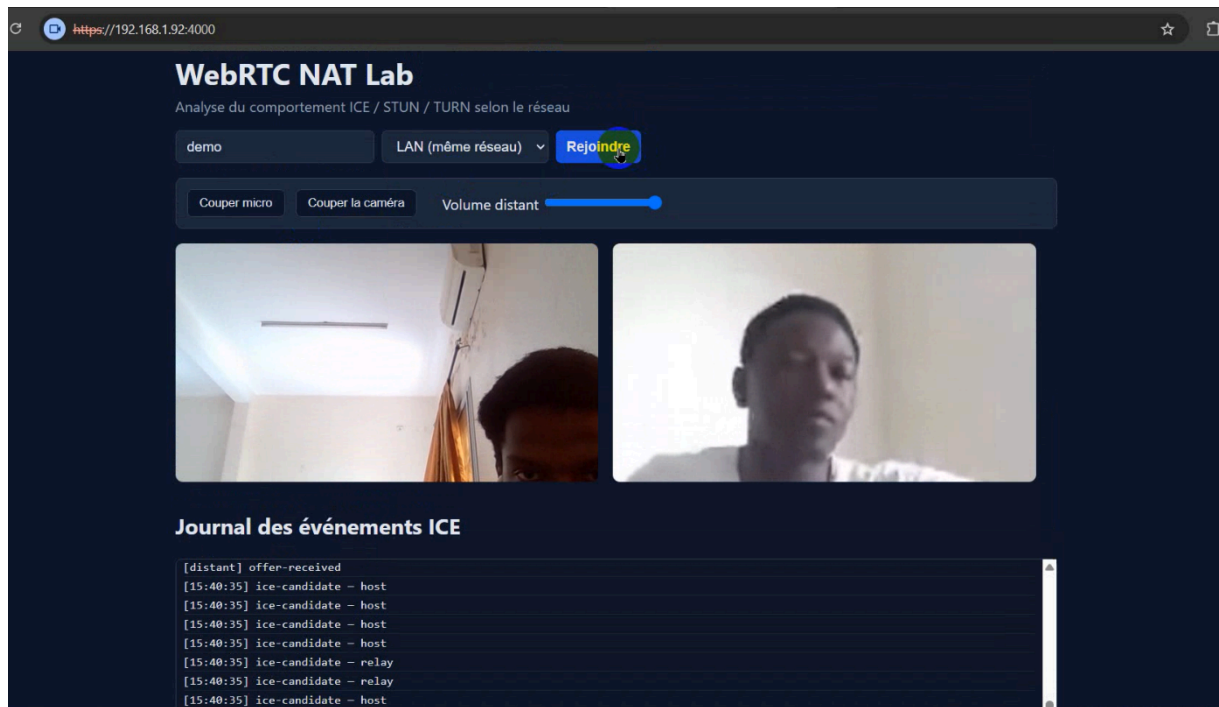
Impossible de vérifier sur le serveur qu'il s'agit bien du domaine **192.168.1.92**, car son certificat de sécurité n'est pas considéré comme fiable par le système d'exploitation de votre ordinateur. Cela peut être dû à une mauvaise configuration ou bien à l'interception de votre connexion par un pirate informatique.

[Continuer vers le site 192.168.1.92 \(dangereux\)](#)

Même avertissement, développé via « Paramètres avancés » : le navigateur explique qu'il ne peut pas vérifier l'identité du serveur, ce qui est le comportement attendu pour un certificat de test non signé par une autorité reconnue.



En haut, une maquette de l'écran de connexion avec le réseau « Wi-Fi public » sélectionné. En bas, l'application réellement fonctionnelle, testée avec l'option « LAN (même réseau) », affichant les flux vidéo local et distant ainsi que le début du journal des évènements ICE.



Application en fonctionnement avec deux flux vidéo actifs et son bandeau de contrôles (micro, caméra, volume distant) visible.

WebRTC NAT Lab

Analyse du comportement ICE / STUN / TURN selon le réseau

demo 4G / Mobile Rejoindre

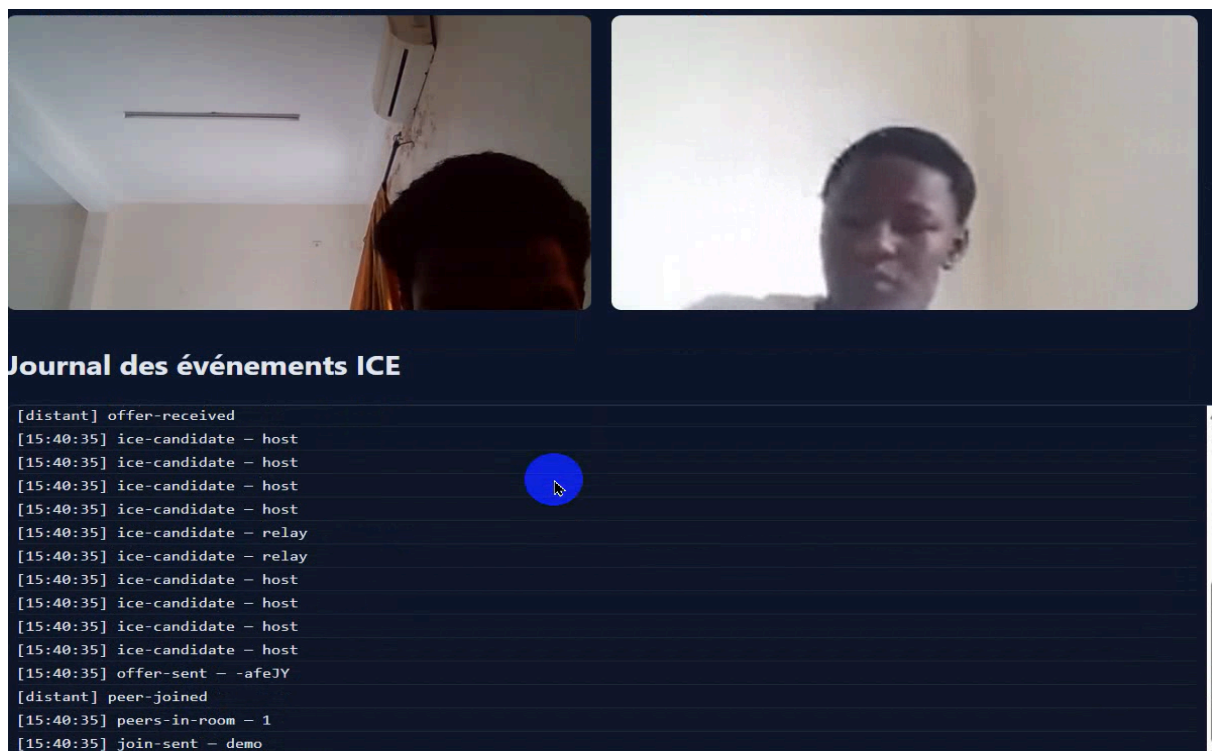
Couper le micro Couper la caméra Volume distant



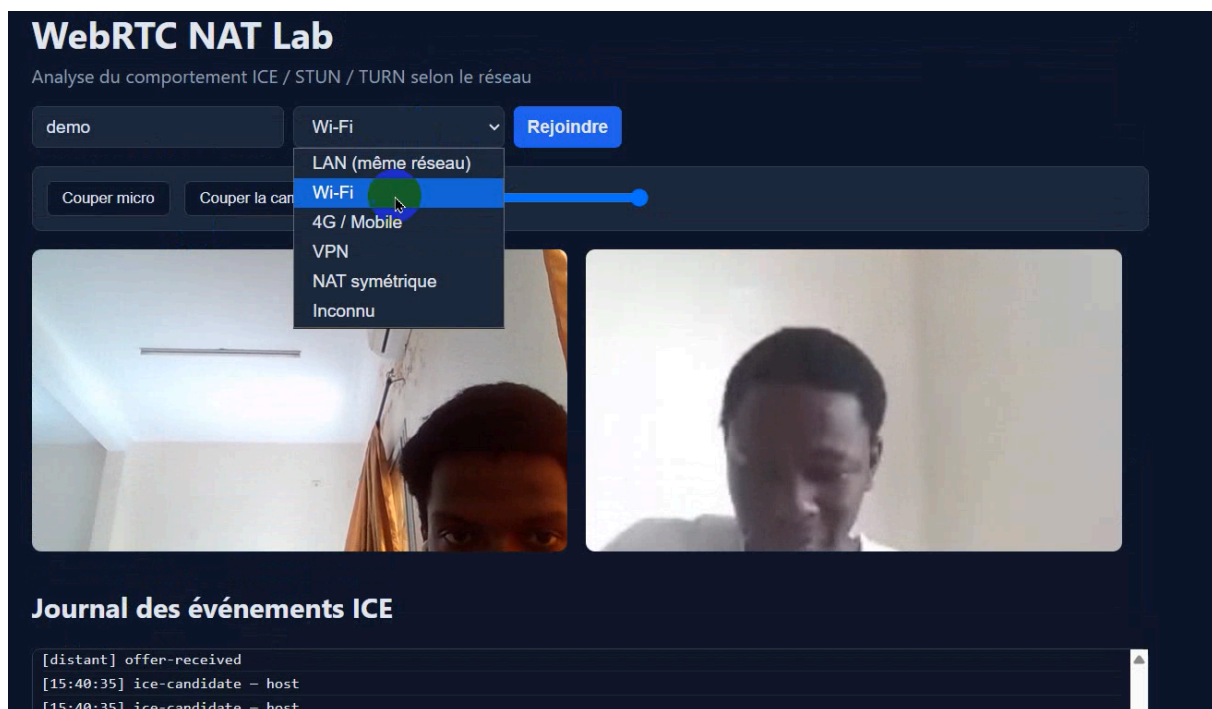
Journal des événements ICE

```
[distant] pc-state
[15:40:36] pc-state - connected
[distant] ice-state
[15:40:36] ice-candidate - end
[15:40:36] ice-state - connected
[distant] ice-candidate - end
```

Test avec le réseau « 4G / Mobile » sélectionné dans le menu : le journal indique que l'état de la connexion (pc-state) comme celui d'ICE (ice-state) sont passés à « connected ».



Vue complète de l'application avec un journal ICE plus détaillé : plusieurs candidats de type host et relay apparaissent, mais aucun candidat srflx, ce qui suggère que les deux postes de test se trouvaient en réalité sur le même réseau local au moment de cette capture.



Menu déroulant de sélection du type de réseau ouvert, montrant l'ensemble des options disponibles pour étiqueter manuellement chaque test (LAN, Wi-Fi, 4G/Mobile, VPN, NAT symétrique, Inconnu).

Testé sur la console pour voir si le micro et la vidéo est autorisée

```
allow pasting
> navigator.mediaDevices.getUserMedia({ video: true, audio:
true })
  .then(s => console.log("✅ SUCCÈS :",
s.getTracks().map(t => t.kind)))
  .catch(e => console.error("❌ ÉCHEC :", e.name, "-",
e.message));
< ▶ Promise {<pending>}
✅ SUCCÈS : ▼ (2) ['audio', 'video'] ⓘ VM344:2
    0: "audio"
    1: "video"
    length: 2
    ▶ [[Prototype]]: Array(0)
> typeof io
< 'function'
> typeof localStream
< 'object'
> |
```

Vérification manuelle, dans la console du navigateur, que l'accès à la caméra et au micro (getUserMedia) est bien autorisé avant de lancer les tests applicatifs.

CONCLUSION

Ce travail a permis de poser un cadre théorique complet pour comprendre comment WebRTC, malgré une ambition architecturale pair-à-pair, doit composer avec la réalité du cloisonnement des réseaux modernes. L'étude des différents comportements de NAT full cone, restricted cone, port-restricted cone et symétrique a montré qu'aucune solution unique ne pouvait garantir la traversée dans tous les cas de figure, ce qui justifie l'existence d'une architecture combinée reposant sur STUN, TURN et ICE.

STUN apparaît comme un mécanisme léger et peu coûteux, particulièrement efficace face aux NAT de type cone, mais insuffisant face aux configurations symétriques. TURN, en assumant le rôle de relais actif du trafic média, garantit à l'inverse un taux de réussite quasi systématique, au prix d'une latence accrue et d'un coût d'infrastructure plus élevé. ICE, enfin, ne se substitue à aucun des deux, mais orchestre leur usage conjoint pour toujours privilégier la voie la plus directe et la plus performante disponible.

L'application de chat vidéo développée dans le cadre de ce projet, associée à son système de journalisation en temps réel, constitue l'instrument permettant de transformer ce cadre théorique en observation empirique : en exposant explicitement le type de candidat retenu pour chaque session, elle rend visible un mécanisme qui reste, dans la plupart des applications grand public, totalement transparent pour l'utilisateur final.

La méthodologie proposée tests croisés sur Wi-Fi public, partage de connexion mobile et réseau universitaire bridé devrait permettre de confirmer, par la mesure du taux de réussite en P2P direct versus le recours au relais TURN, l'hypothèse centrale de cette étude : la fiabilité de la connexion directe WebRTC dépend étroitement de la nature du NAT et du pare-feu traversés, et le recours à un serveur relais TURN demeure, en l'état actuel des infrastructures réseau (généralisation du CGNAT, durcissement des politiques de sécurité institutionnelles), une nécessité pratique incontournable plutôt qu'une simple solution de secours marginale.

Au-delà du cas spécifique de ce projet, cette analyse illustre plus largement un enjeu central des architectures réseau contemporaines : concilier les impératifs de sécurité et de gestion de l'adressage IP portés par le NAT et les pare-feux avec les exigences de connectivité directe et de faible latence qu'imposent les usages temps réel. Les perspectives d'approfondissement de

ce travail pourraient inclure l'élargissement du panel de réseaux testés, l'intégration de métriques qualitatives (latence, gigue, perte de paquets) en complément du seul taux de réussite, ou encore l'étude du comportement d'IPv6, qui, en supprimant à terme la nécessité du NAT, pourrait rebattre significativement les termes de cette problématique.